SafeMAP: Analysis-Guided Planning and Validation for Safe C-to-Rust Translation

1st Jacob Sebastian Cyril
Department of Computer Science
CHRIST (Deemed to be University)
Bangalore, India
jacob.sebastian@mca.christuniversity.in

2nd Benita Jaison
Department of Computer Science
CHRIST (Deemed to be University)
Bangalore, India
email address or ORCID

3rd New Begin M.
Department of Computer Science
CHRIST (Deemed to be University)
Bangalore, India
email address or ORCID

***Abstract*—Automatic C-to-Rust translators often preserve C pointer conventions and emit unsafe Rust. SafeMAP takes a different approach. It records each migration decision, from candidate classification through behavioral testing, in an auditable plan. Its analyzer collects pointer and dependency facts. A deterministic synthesizer emits Rust only when a safe rule matches exactly. Accepted code must compile without unsafe operations, expose no raw pointers in its public API, and match the original C on recorded randomized inputs. The C oracle runs under AddressSanitizer and UndefinedBehaviorSanitizer. Six helpers from a 32-function LLVM development corpus pass every check, including 1,000 differential cases per generated project. We specify a frozen held-out study on three independently maintained C libraries. The protocol uses outcome-blind human labels, records analyzer provenance, compares against direct LLM translation to safe Rust, and tests individual components through ablation. At manuscript freeze, held-out outcomes and reviewer agreement replace this protocol-only statement. Development results test the mechanism but do not measure generalization.**

***Index Terms*—C-to-Rust migration, safe Rust, static analysis, program translation, software maintenance, memory safety**

I. Introduction

C remains common in systems software. Its memory model leaves programmers responsible for preventing buffer overflows, use-after-free errors, null dereferences, and data races. Safe Rust checks ownership and borrowing at compile time while retaining low-level control. Translation is difficult because many C idioms have no direct safe-Rust equivalent. Raw pointers, output parameters, manual allocation, nullable values, and integer error codes require changes in representation and API design.

The established C2Rust translator preserves C behavior by generating Rust that often uses raw pointers and unsafe operations [1]. Its output is a useful reference, although it does not provide the guarantees of safe Rust. Prior work uses static analysis, refactoring, ownership-guided rewriting, transpiler redesign, and LLM-based rewriting to remove unsafe code after translation [2]–[8]. Other studies ask whether LLMs can translate directly to idiomatic Rust and repair failures with compiler or test feedback [9]–[13]. A third line targets safe Rust directly for structured subsets of C [14].

SafeMAP also targets safe Rust, but it is an experimental migration pipeline, not a general C compiler. Its acceptance rule is strict. Reducing unsafe code is insufficient, as is compilation with unsafe blocks or a C-style raw-pointer API. SafeMAP accepts a unit only when the generated Rust satisfies the project's safe-code policy and passes every applicable check. If no supported safe path exists, the tool rejects the unit and records why.

We ask:

- **RQ1:** For a restricted, statically identifiable subset of C idioms, how often can SafeMAP synthesize fully safe Rust?
- **RQ2:** Which C idioms are currently handled reliably by deterministic safe synthesis, and which idioms remain limiting?
- **RQ3:** How does SafeMAP compare with direct LLM safe translation and raw C2Rust under one acceptance policy?
- **RQ4:** Which individual planning and validation components contribute to accepted translation?
- **RQ5:** Which artifacts are required to audit and reproduce a safe-first migration decision?

The evaluation keeps development and test data separate. Development data contains 40 authored microbenchmarks, six authored regression modules, and ten LLVM programs whose results were already known. The frozen test corpus retains every function definition in preselected production source files from pinned revisions of inih, cJSON, and libcsv. The files stay sealed until the implementation is frozen.

This paper contributes:

- a C-to-Rust pipeline that accepts code only when it contains no unsafe Rust and exposes no raw pointers through its public API;
- a migration plan that records detected C idioms, proposed Rust APIs, eligibility decisions, and required checks;
- a deterministic synthesizer for supported C idioms, connected to compilation, source-policy checks, differential testing, and recorded sanitizer status; and
- a pre-specified function-level study with independent eligibility labels, analyzer provenance, replayable test cases, component ablations, and a direct safe-output baseline.

II. Background and Related Work

A. C2Rust and safer-Rust refactoring

C2Rust translates C through a mostly syntactic process and is often used as a baseline or a migration starting point [1].

It preserves many C constructs with raw pointers and unsafe operations, so much of its output falls outside Rust's safe subset. Emre et al. found that raw pointers cause much of this unsafety [3]. Hong used static analysis to replace selected unsafe constructs in C2Rust output, including lock APIs and output parameters [4].

C2SaferRust combines C2Rust, static analysis, and LLM rewriting to reduce unsafe code in translated real-world programs [6]. Crown infers ownership models for C pointers and converts many of them to safe Rust types [5]. CRustS and Cp2SRust use different source-to-source transformations and transpiler designs [2], [7]. Industry studies and surveys report that automatic translation still needs substantial manual work and testing [8], [15]. SafeMAP uses C2Rust as a reference lane. Its primary lane generates final safe Rust directly for eligible units. Reductions in unsafe code remain diagnostic measurements; they count as migrations only when the final output passes the stated acceptance policy.

B. LLM-based C-to-Rust translation

LLMs can produce idiomatic Rust, although correctness and type migration remain open problems. Hong and Ryu studied an LLM translator supplied with candidate signatures, caller and callee context, and compiler feedback [9]. C2RustTV uses LLMs for production and test-code translation, then checks the result with generated tests and repair loops [10]. Flourine evaluates LLM translation through cross-language differential fuzzing and finds lower success on more complex real-world samples [11]. RustFlow studies prompting and iterative repair [13]. Valenzuela et al. examine compiler errors in direct C-to-safe-Rust translation and in LLM repair of C2Rust output [12]. RustMap combines program analysis and LLM support at project scale [16].

C. Safe-Rust translation for C subsets

Scylla translates an applicative C subset to safe Rust and shows how source structure can make safe generation possible [14]. SafeMAP likewise makes no claim that arbitrary C can be migrated automatically. It is a small research prototype built around migration plans, recorded rejections, and reproducible acceptance measurements.

D. Validation and undefined behavior

Compilation alone cannot establish a safe migration. SafeMAP records compilation status, remaining unsafe constructs, raw pointers in public APIs, differential test results, C sanitizer results, and Miri status. Miri can detect undefined behavior during Rust execution [17]. ASan and UBSan check the executed C cases before SafeMAP uses them as behavioral oracles [18], [19]. Miri is disabled in the frozen experiment because the generated crates and installed toolchain do not provide a uniform applicable oracle; the artifact reports it as skipped rather than passed.

III. SafeMAP design

A. Outcome labels

SafeMAP records candidate classification and implementation support separately. The candidate label is `candidate_safe`, `manual_refactor_required`, `unsafe_required`, or `unknown`. A candidate-safe label means that screening found no known obstacle. It does not prove that a safe translation exists. The separate `implemented_support` field is true only when an exact synthesis rule matches. Later fields record generation, compilation, policy compliance, and behavioral test results. Every decision also identifies whether its facts came from libclang or the regular-expression fallback.

B. Acceptance policy

SafeMAP reports policy and behavioral acceptance separately. A policy-safe unit must meet these conditions:

1) The final Rust code compiles with `#![forbid(unsafe_code)]`.
2) The final Rust contains no `unsafe` blocks, `unsafe fn`, `unsafe impl`, or `extern "C"` blocks.
3) The public API does not expose raw pointers such as `*const T` or `*mut T`.
4) The planned function or module unit is present in the final Rust output.

A behaviorally validated unit is policy-safe and passes an applicable C-versus-Rust oracle. A skipped or inapplicable check does not count as a pass. Cargo test and Clippy must pass when enabled. The frozen run records Miri as disabled and never treats that status as a pass. ASan and UBSan monitor the C oracle; a sanitizer finding disqualifies that execution as a correctness oracle. Partially migrated C2Rust output may still aid manual work, but SafeMAP does not accept it unless it passes the same policy.

C. Pipeline

SafeMAP processes each C input in seven stages. Figure 1 shows both execution lanes and the policy they share:

1) Locate source files and recover compile information when possible.
2) Identify functions, signatures, pointer use, allocation patterns, unsupported constructs, and dependency structure.
3) Assign the screening taxonomy without consulting implemented synthesis rules.
4) Classify migration idioms such as output parameters, pointer-length arrays, nullable pointers, C strings, and return-code-plus-output patterns.
5) Build a plan that records the target Rust signature, ownership model, validation requirements, and reasons for any rejection.
6) Generate Rust when a deterministic rule supports the planned pattern.
7) Compile the result, check remaining unsafe code, count raw pointers, run differential tests, check ASan/UBSan on C, record Miri status, and export function-level measurements.

D. Worked plan and rule selection

Consider the LLVM development helper:

```
double loop(float *x, float *y, long n) {
  double acc = 0.0;
  for (long i=0; i<n; ++i)
    acc += (double)x[i] * (double)y[i];
  return acc;
}
```

libclang finds two indexed pointers paired with `n`. It finds no indexed writes, calls, or dependencies. The classifier returns `candidate_safe`. The plan maps each pointer-length pair to a shared slice, and the `slice_dot_product` rule provides `implemented_support`. Since `loop` is a Rust keyword, the generated function uses a raw identifier:

```
pub fn r#loop(x: &[f32], y: &[f32]) -> f64 {
  x.iter().zip(y.iter())
   .map(|(&a,&b)| (a as f64)*(b as f64))
   .fold(0.0_f64, |s,v| s + v)
}
```

The initial value `+0.0` is necessary. Differential testing showed that a generic floating-point `sum()` could disagree with C on the sign bit of zero. The generated helper matches C bit for bit on 1,000 recorded cases and passes the Cargo, Clippy, ASan, and UBSan checks. LLVM's `doTest`, by comparison, calls allocation and memory APIs outside the supported subset. The classifier labels it `unsafe_required`, so synthesis does not run.

```
facts <- analyze(function)
candidate <- classify(facts)
unit <- SCC(call_graph, function)
plan <- safe_signature(facts) + idiom_facts
if candidate != candidate_safe:
    support <- not_applicable
else if exact_rule_matches(function, plan):
    support <- implemented_support
else: support <- not_implemented
generate only with implemented_support
report policy and behavior separately
```

E. Supported idioms

The deterministic synthesizer currently handles: integer and floating scalar expressions, integer boolean and bit-reversal idioms, pointer-length and fixed-size arrays, scalar structs and field access, dependency-ordered internal helper calls, simple entry-point composition, mutable scalar pointer updates, output parameters, multiple output parameters, return-code plus output-parameter to `Result`, nullable pointers, reductions including two-slice floating dot products, simple C string lengths, simple allocation-to-`Vec` patterns, and single owned allocations to `Box`. It does not support unions, inline assembly, volatile memory access, unresolved aliasing, complex macros, custom allocators, function pointers, or pointer-integer casts.

F. Baselines

SafeMAP compares its primary lane with two baselines. Raw C2Rust measures how much unmodified output passes the policy. The second baseline asks a fixed LLM to translate C directly to safe Rust without SafeMAP facts or plans. The frozen baseline is Google's `gemini-3.5-flash-lite`, accessed through its OpenAI-compatible API at temperature zero. The experiment keeps each prompt and response along with model settings, token counts, retries, and failures. It makes one request per project and performs no automatic retry; quota or provider failures remain in the denominator. Five ablations remove one component at a time: pointer-role classification, safe-signature generation, dependency grouping, idiom plans, or validation feedback. Every lane uses the same source units and behavioral checks.

IV. Implementation

SafeMAP is a Python package. Separate modules handle ingestion, static analysis, migration planning, translation, repair, validation, and measurement. For a supported idiom, the deterministic synthesizer emits Rust under `#![forbid(unsafe_code)]`. The validation layer derives scalar and array harnesses from function signatures. It saves seeds and concrete C/Rust inputs and outputs, then records the result of each compiler, policy, differential, sanitizer, Clippy, and recorded Miri status. The benchmark runner writes CSV, Markdown, and LaTeX tables. One evaluation command combines SafeMAP, case-study, external-corpus, C2Rust, and ablation results. Its CSV files also record function roles, complete-project outcomes, outcome stages, and the status of each check.

The analyzer uses libclang when available. Otherwise it uses a pattern-based parser and records why the fallback was needed. Dependency analysis groups mutually dependent functions into one unit. Classification and planning finish before synthesis begins. The deterministic lane needs neither C2Rust nor a network model service.

V. Evaluation methodology

A. Datasets

The development corpus contains 40 authored microbenchmarks, six authored modules, and ten LLVM test-suite programs used during earlier experiments. The LLVM sources are pinned at commit `6cdc54e` [20]. They contain 589 lines of C and 32 functions. libclang analyzed every function. Their outcomes informed rule development, so they provide no held-out evidence.

The sealed test corpus uses pinned production sources from inih r62, cJSON v1.7.19, and libcsv commit `b1d5212`. A documented selection rule keeps every function in named production `.c` files without consulting SafeMAP output. Before execution, the experiment records repository URLs, revisions, source hashes, licenses, and physical line counts. A materialization script blocks access until the implementation has a frozen Git commit. This keeps test sources out of rule development.

Two reviewers label each held-out function without seeing the SafeMAP decision. They then resolve disagreements. The labels use the candidate taxonomy. `unknown` cases are reported separately and excluded from binary precision and recall. The study reports raw agreement, Cohen's κ, disagreements by construct, and separate results for libclang and the fallback backend.

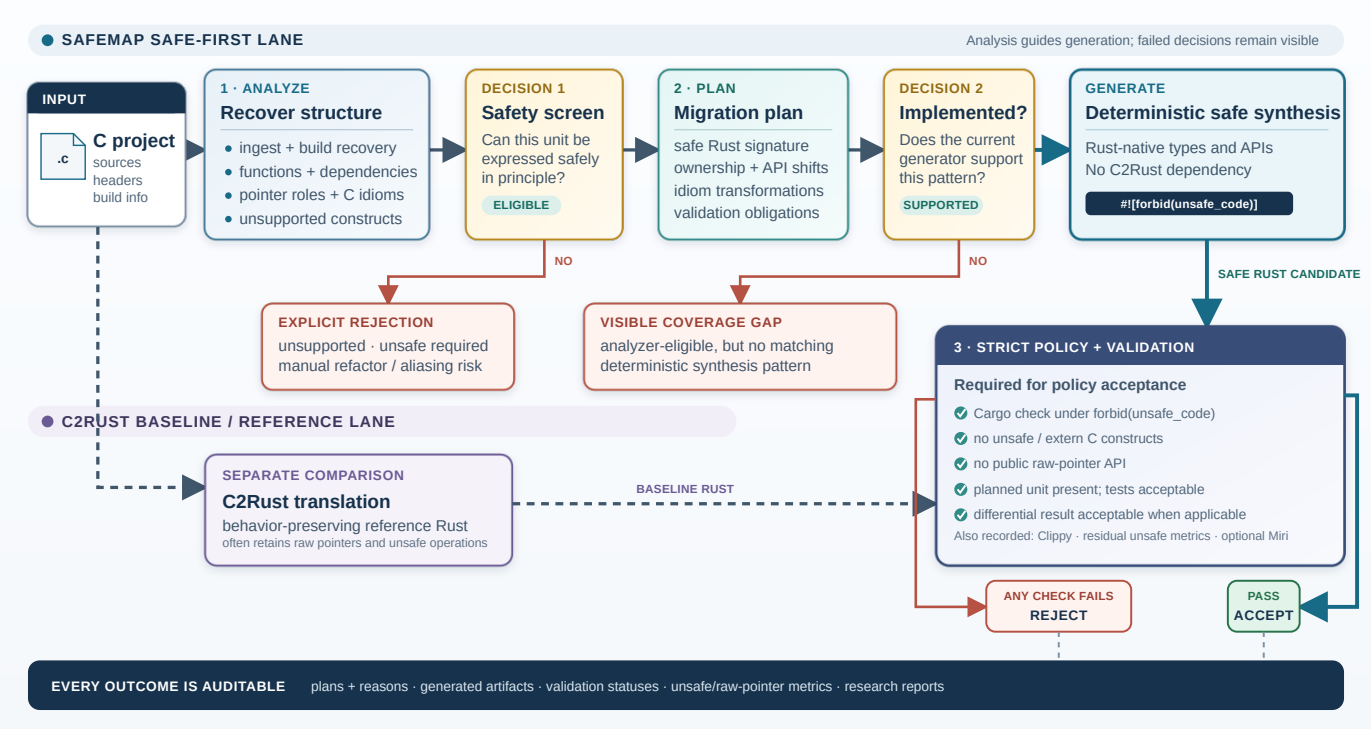

Fig. 1. SafeMAP conversion and acceptance pipeline. Static analysis records eligibility separately from synthesis support. The primary lane generates Rust from a migration plan; C2Rust runs in a separate reference lane. The same policy applies to both outputs. Reports retain unsupported cases, missing synthesis rules, and failed checks as distinct outcomes.

B. Metrics

The primary effectiveness measure is the number of behaviorally validated, policy-safe units divided by all retained held-out units. The report also gives the count at each earlier stage: candidate-safe, supported, generated, compiling, and policy-safe. Results are separated by declared targets, helpers, entry points, other functions, and complete modules. Analyzer measurements include eligibility precision and error counts. Corpus tables report project and function counts, lines of code, cyclomatic complexity, pointer density, and construct frequencies.

C. Execution snapshot

Before unsealing, the implementation and configuration are committed. A freeze record binds the experiment to that Git revision. The held-out test then runs once. Artifacts record source and configuration hashes, tool versions, analyzer backend, commands, generated code, differential inputs and outputs, random seeds, sanitizer status, timing, and failures. The direct-LLM baseline fixes the model and sampling settings before this run. The implementation-freeze record stores the exact pre-freeze test command and result; the test count is not a held-out outcome.

VI. Results

A. Development evidence

The LLVM development corpus contains 32 functions from ten projects. After two analysis fixes and

TABLE I
DEVELOPMENT-ONLY OUTCOME CASCADE. "CAND." DENOTES `CANDIDATE_SAFE`; "BEHAV." DENOTES BEHAVIORALLY VALIDATED POLICY-SAFE OUTPUT.

Corpus	Total	Cand.	Support	Gen.	Compile	Policy	Behav.
LLVM programs	32	21	6	6	6	6	6
Microbenchmarks	81	76	36	36	36	36	36
Authored modules	25	25	19	19	19	19	17

rule changes based only on development data, 21 functions receive `candidate_safe` and six receive `implemented_support`. The six generated functions compile, satisfy the policy, and pass behavioral testing. They are `sqr`, `fade`, `lerp`, the 32-bit and 64-bit reversal helpers, and the two-slice floating-point reduction. Before these changes, only `sqr` passed. The 6/32 result measures development progress and exercises the reporting pipeline. It cannot support a generalization claim because the same corpus guided the rule changes.

Two bugs show why the report keeps intermediate outcomes. The pointer screen first treated indexed reads as writes, which produced a false mutable-alias conflict for the floating-point reduction. It also parsed >> as a comparison and gave bit reversal a Boolean return type. Both bugs now have regression tests. Exact differential comparison found the separate signed-zero error described earlier.

On the 40 authored development programs, the full configuration validates 36 of 81 units, equal to 36 of 76 candidate-

TABLE II
BEHAVIORALLY VALIDATED UNITS IN COMPONENT ABLATIONS. DENOMINATORS INCLUDE EVERY FUNCTION OR DEPENDENCY UNIT IN EACH DEVELOPMENT CORPUS.

Configuration	Microbenchmarks	Modules
Full system	36/81	17/25
Without pointer roles	16/81	3/25
Without safe signatures	6/81	0/25
Without dependency grouping	36/81	15/25
Without idiom plans	6/81	1/25
Without validation feedback	36/81	17/25

safe units, and 35 of 40 declared targets. Without pointer roles, the counts fall to 16/81 units and 15/40 targets. Without safe signatures or idiom plans, they fall to 6/81 and 5/40. Removing dependency grouping or validation feedback has no effect on this suite. These development ablations are diagnostic. The zero changes for the last two components show that the authored programs do not test their proposed benefits, so no causal conclusion follows.

A sixth authored module tests fixed arrays, a scalar struct, field access, two internal calls, five functions in one crate, and a small `main`. All five functions compile, pass policy checks, and pass the generated differential and sanitizer harness. Across six authored modules, the full system validates 17/25 units. Removing dependency grouping reduces the result to 15/25 because the two composed callers lose support. The module was written to test this mechanism, so the result is development evidence.

B. Held-out evaluation status

The held-out corpus has not been materialized or run for this manuscript. There is therefore no effectiveness estimate for RQ1–RQ4. We also do not combine authored and LLVM development results into a substitute test rate. After the implementation freeze, the artifact will generate final tables for corpus composition, label agreement, eligibility errors, outcomes by function role, complete modules, both baselines, and the five component ablations. Those results must replace this section before submission.

VII. DISCUSSION

A. How the acceptance rule changes comparison

Code that compiles while exposing raw pointers through its public API has not reached safe Rust. The same policy therefore applies to every experimental lane. This prevents raw C2Rust output from being counted beside policy-safe output merely because both compile. C2Rust remains useful as a reference, and its output passes only when it meets the shared policy.

B. Isolating planning components

The synthesizer requires a plan, so removing every form of guidance guarantees failure. The main ablation instead removes one input per run: pointer roles, safe signatures, dependency grouping, idiom plans, or validation feedback. A no-guidance run remains as a dependency check. Source units and validators stay fixed across runs.

C. Recorded rejection

Unsupported constructs such as unions, volatile access, inline assembly, and function pointers occur in real C programs. Partial output can hide their effect on a migration. SafeMAP records each unsupported case as an outcome. The record shows where automatic processing stopped and where manual review must begin.

D. Eligibility and synthesis coverage

The authored suites contain deliberately supported patterns, so they measure regression coverage. In the LLVM development set, 15 of 21 candidate-safe functions still lack a synthesis rule. Candidate labels and implementation support must therefore remain separate. Held-out human labels are needed to tell how much of the gap comes from classification error and how much comes from missing rules.

VIII. THREATS TO VALIDITY

Benchmark representativeness. The authored and LLVM corpora are development data. The held-out set improves project diversity but contains three small C libraries instead of large applications. We report size, complexity, pointer use, and construct frequencies so readers can judge the scope. The results do not cover arbitrary C.

Implementation maturity. SafeMAP is a research prototype. The analyzer and synthesizer use patterns and do not implement full C semantics. The tool reports unsupported constructs, and its supported subset remains small.

Differential testing limits. Differential testing over bounded random inputs is evidence, not equivalence proof. Generated harnesses cover supported scalar and array shapes. They do not cover every heap, callback, or stateful API. Saved seeds and outputs allow replay. ASan and UBSan reduce the chance of using an undefined C execution as an oracle, but they cannot say whether unexecuted inputs are defined.

Label subjectivity. Candidate safety sometimes requires judgment about whether an API redesign is automatic or manual. Two outcome-blind labels, retained disagreements, adjudication, confidence scores, and κ expose that judgment in the report.

Toolchain sensitivity. C2Rust, Clang, Rust, Clippy, and Miri can behave differently across host versions and configurations. The artifact records skipped and failed stages. Reproduction across machines will need tighter toolchain pinning.

Metric interpretation. Units are functions or dependency groups, not lines of code or whole-project migration percentages. Analyzer eligibility may exceed implemented synthesis support. Dataset size and complexity describe the corpus but do not remove this measurement limit.

IX. Future work

Current rules handle fixed arrays, scalar structs and fields, internal calls, small entry-point compositions, and multi-function crates. Planned work covers stateful APIs, callbacks, nested aggregates, unions with safe tagged representations, and aliases that require flow-sensitive borrowing analysis. Test generation also needs structured inputs, buffers with semantic length constraints, and preconditions derived from metadata. If held-out results lead to rule changes, the next generalization study will need a new frozen corpus.

X. Conclusion

SafeMAP records each stage of C-to-Rust migration instead of reducing the result to compilation success. Classification, synthesis support, policy checks, and behavioral tests have separate outcomes. A missing test harness cannot count as a behavioral pass. Six functions in the LLVM development corpus now pass every stage, among them a floating-point reduction and two bit-reversal helpers. The held-out protocol is prepared for three independent projects and includes outcome-blind labels, replayable differential cases, C sanitizers, a direct safe-output baseline, and component ablations. Until that study runs, the results establish only that the implemented rules and measurement pipeline work on development data.

References

[1] Immunant, "C2Rust: C to rust translator," https://github.com/immunant/c2rust, accessed 2026-07-28.

[2] M. Ling, Y. Yu, H. Wu, Y. Wang, J. R. Cordy, and A. E. Hassan, "In rust we trust: A transpiler from unsafe c to safer rust," in *2022 IEEE/ACM 44th International Conference on Software Engineering: Companion Proceedings*, 2022, pp. 354–355.

[3] M. Emre, R. Schroeder, K. Dewey, and B. Hardekopf, "Translating c to safer rust," *Proceedings of the ACM on Programming Languages*, vol. 5, no. OOPSLA, 2021.

[4] J. Hong, "Improving automatic c-to-rust translation with static analysis," in *2023 IEEE/ACM 45th International Conference on Software Engineering: Companion Proceedings*, 2023, pp. 273–277.

[5] H. Zhang, C. David, Y. Yu, and M. Wang, "Ownership guided c to rust translation," in *Computer Aided Verification*, ser. Lecture Notes in Computer Science, vol. 13966, 2023, pp. 459–482.

[6] V. Nitin, R. Krishna, L. L. d. Valle, and B. Ray, "C2SaferRust: Transforming c projects into safer rust with neurosymbolic techniques," *IEEE Transactions on Software Engineering*, vol. 52, no. 2, pp. 618–630, 2026.

[7] V. Mohapatra, D. Tripuramallu, A. K. Behera, S. Pinisetty, S. A. Shivaji, and A. Bandeppa, "Cp2SRust: A transpiler for c/c++ to safer rust," in *Innovations in Software Engineering Conference*, 2026, pp. 1–10.

[8] J. Hong and S. Ryu, "Automatically translating c to rust," *Communications of the ACM*, vol. 68, no. 11, pp. 58–65, 2025.

[9] ——, "Type-migrating c-to-rust translation using a large language model," *Empirical Software Engineering*, vol. 30, no. 1, 2025.

[10] H. Zhou, Y. Luo, M. Zhang, and D. Xu, "C2RustTV: An llm-based framework for c to rust translation and validation," in *2025 IEEE 49th Annual Computers, Software, and Applications Conference*, 2025, pp. 1254–1259.

[11] H. F. Eniser, H. Zhang, C. David, M. Wang, M. Christakis, B. Paulsen, J. Dodds, and D. Kroening, "Towards translating real-world code with llms: A study of translating to rust," 2025.

[12] A. Valenzuela, M. Gonzalez-Mallo, C. Gutierrez, D. Garcia-Gasulla, G. Kestor, and S. Royuela, "From c to rust: Evaluating llm capabilities in transpilation through compilation errors," in *ISC High Performance 2025 International Workshops*, ser. Lecture Notes in Computer Science, vol. 16091, 2025, pp. 311–324.

[13] R. Zhang, S. Zhang, and L. Xie, "A systematic exploration of c-to-rust code translation based on large language models: Prompt strategies and automated repair," *Automated Software Engineering*, vol. 33, no. 1, 2026.

[14] A. Fromherz and J. Protzenko, "Scylla: Translating an applicative subset of c to safe rust," *Proceedings of the ACM on Programming Languages*, vol. 10, no. OOPSLA1, pp. 823–849, 2026.

[15] E. Viirola, J. Hummel, E. Söderberg, A. Bexell, P. Kornevi, and U. Asklund, "From c to rust – how feasible is it?" in *Product-Focused Software Process Improvement*, ser. Lecture Notes in Computer Science, vol. 16362, 2025, pp. 19–35.

[16] X. Cai, J. Liu, X. Huang, Y. Yu, H. Wu, C. Li, B. Wang, I. N. B. Yusuf, and L. Jiang, "RustMap: Towards project-scale c-to-rust migration via program analysis and llm," in *Engineering of Complex Computer Systems: 29th International Conference, ICECCS 2025*, 2025, pp. 283–302.

[17] R. Jung, B. Kimock, C. Poveda, E. Sánchez Muñoz, O. Scherer, and Q. Wang, "Miri: Practical undefined behavior detection for rust," *Proceedings of the ACM on Programming Languages*, vol. 10, no. POPL, 2026.

[18] K. Serebryany, D. Bruening, A. Potapenko, and D. Vyukov, "AddressSanitizer: A fast address sanity checker," in *2012 USENIX Annual Technical Conference*. USENIX Association, 2012, pp. 309–318. [Online]. Available: https://www.usenix.org/conference/atc12/technical-sessions/presentation/serebryany

[19] LLVM Project, "UndefinedBehaviorSanitizer," https://clang.llvm.org/docs/UndefinedBehaviorSanitizer.html, accessed 2026-07-28.

[20] ——, "LLVM test suite guide," https://llvm.org/docs/TestSuiteGuide.html, 2026, accessed 2026-07-23; corpus pinned to commit 6cdc54e005552e3444fa7402cd18a6e4b6db195d.